

Database Principles

CCS3400

Chapter 07: Database Objects

Learning Objectives

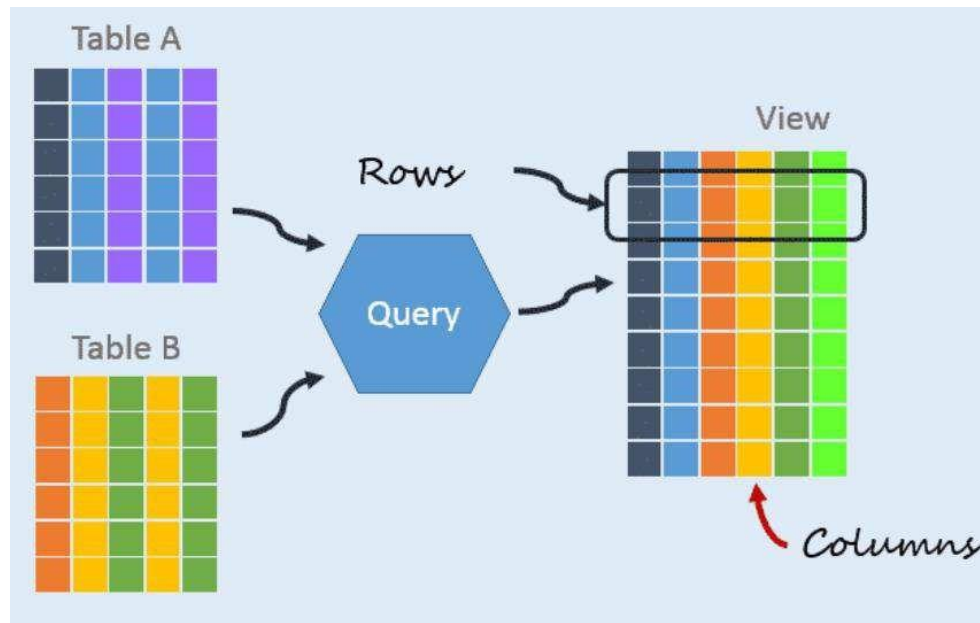
- After completing this lesson, you should be able to do the following:
 - Create simple and complex views
 - Retrieve data from views
 - Create, maintain, and use sequences
 - Create and maintain indexes

Database Objects

Object	Description
Table	Basic unit of storage; composed of rows and columns
View	Logically represents subsets of data from one or more tables
Sequence	Generates primary key values
Index	Improves the performance of some queries
Synonym	Alternative name for an object

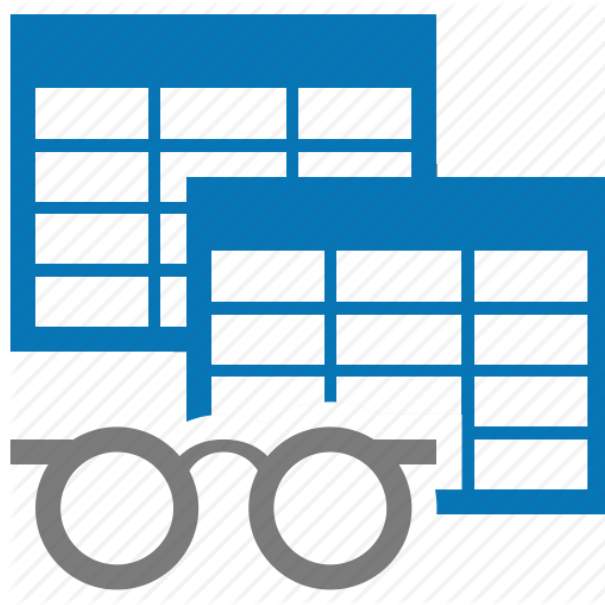
What is View?

- **View** is a virtual table based on the result-set of an **SQL** statement



Advantages of View

- To restrict data access
- To provide data independence
- To make complex queries easy
- To present different views of the same data



Simple Views and Complex Views

Feature	Complex	Simple
Number of Tables	One or More	One
Contain functions	Yes	No
Contain groups of data	Yes	No
DML operation	Not always	Yes

Creating a View

```
CREATE [OR REPLACE] [FORCE|NOFORCE] VIEW view
    [(alias[, alias]...)]
    AS subquery
    [WITH CHECK OPTION [CONSTRAINT constraint]]
    [WITH READ ONLY [CONSTRAINT constraint]];
```

You can create a view by embedding a subquery in the CREATE VIEW statement.

In the syntax:

OR REPLACE

Re-creates the view if it already exists

FORCE

Creates the view regardless of whether or not the base tables exist

NOFORCE

Creates the view only if the base tables exist (This is the default.)

view

Is the name of the view

alias

Specifies names for the expressions selected by the view's query
(The number of aliases must match the number of expressions
selected by the view.)

subquery

Is a complete SELECT statement (You can use aliases for the
columns in the SELECT list.)

WITH CHECK OPTION

Specifies that only those rows that are accessible to the view can
be inserted or updated

constraint

Is the name assigned to the CHECK OPTION constraint

WITH READ ONLY

ensures that no DML operations can be performed on this view

Creating a View

- Create the EMPVU80 view, which contains details of the employees in department 80:

```
CREATE VIEW empvu80
AS SELECT employee_id, last_name, salary
FROM employees
WHERE department_id = 80;
```

```
CREATE VIEW succeeded.
```

- Describe the structure of the view by using the *iSQL*Plus* DESCRIBE command:

```
DESCRIBE empvu80
```

Creating a View

- Create a view by using column aliases in the subquery:

```
CREATE VIEW    salvu50
  AS SELECT    employee_id ID_NUMBER, last_name NAME,
              salary*12 ANN_SALARY
    FROM      employees
    WHERE      department_id = 50;
```

```
CREATE VIEW succeeded.
```

- Select the columns from this view by the given alias names.

Retrieving Data from a View

```
SELECT *  
FROM salvu50;
```

	R Z ID_NUMBER	R Z NAME	R Z ANN_SALARY
1	124	Mourgos	69600
2	141	Rajs	42000
3	142	Davies	37200
4	143	Matos	31200
5	144	Vargas	30000

Modifying View

- Modify the EMPVU80 view by using a CREATE OR REPLACE VIEW clause. Add an alias for each column name:

```
CREATE OR REPLACE VIEW empvu80
  (id_number, name, sal, department_id)
AS SELECT  employee_id, first_name || ' '
           || last_name, salary, department_id
  FROM      employees
  WHERE     department_id = 80;
```

CREATE OR REPLACE VIEW succeeded.

- Column aliases in the CREATE OR REPLACE VIEW clause are listed in the same order as the columns in the subquery.

Creating Complex View

- Create a complex view that contains group functions to display values from two tables:

```
CREATE OR REPLACE VIEW dept_sum_vu
(name, minsal, maxsal, avgsal)
AS SELECT    d.department_name, MIN(e.salary),
            MAX(e.salary), AVG(e.salary)
FROM        employees e JOIN departments d
ON          (e.department_id = d.department_id)
GROUP BY d.department_name;
```

CREATE OR REPLACE VIEW succeeded.

```
SELECT *
FROM    dept_sum_vu;
```

	NAME	MINSAL	MAXSAL	AVGSAL
1	Administration	4400	4400	4400
2	Accounting	8300	12000	10150
3	IT	4200	9000	6400
4	Executive	17000	24000	19333.33333333333333333333...
5	Shipping	2500	5800	3500
6	Sales	8600	11000	10033.33333333333333333333...
7	Marketing	6000	13000	9500

Rules for performing DML Operations on View

- You can usually perform DML operations on simple views. 
- You cannot remove a row if the view contains the following:
 - Group functions
 - A `GROUP BY` clause
 - The `DISTINCT` keyword
 - The pseudocolumn `ROWNUM` keyword

UPDATE Operations on a View

- You cannot modify data in a view if it contains:
 - Group functions
 - A GROUP BY clause
 - The DISTINCT keyword
 - The pseudocolumn ROWNUM keyword
 - Columns defined by expressions

INSERT Operations on a View

- You cannot add data through a view if the view includes:
 - Group functions
 - A GROUP BY clause
 - The DISTINCT keyword
 - The pseudocolumn ROWNUM keyword
 - Columns defined by expressions
 - NOTNULL columns in the base tables that are not selected by the view

Using the WITH CHECK OPTION Clause

- You can ensure that DML operations performed on the view stay in the domain of the view by using the `WITH CHECK OPTION` clause:

```
CREATE OR REPLACE VIEW empvu20
AS SELECT      *
   FROM        employees
   WHERE       department_id = 20
   WITH CHECK OPTION CONSTRAINT empvu20_ck ;
```

```
CREATE OR REPLACE VIEW succeeded.
```

- Any attempt to `INSERT` a row with a `department_id` other than 20, or to `UPDATE` the department number for any row in the view fails because it violates the `WITH CHECK OPTION` constraint.

Using the WITH CHECK OPTION Clause

The `WITH CHECK OPTION` clause specifies that `INSERTs` and `UPDATEs` performed through the view cannot create rows that the view cannot select. Therefore it enables integrity constraints and data validation checks to be enforced on data being inserted or updated. If there is an attempt to perform DML operations on rows that the view has not selected, an error is displayed, along with the constraint name if that has been specified.

```
UPDATE empvu20
SET    department_id = 10
WHERE  employee_id = 201;
```

causes:

Error report:

```
SQL Error: ORA-01402: view WITH CHECK OPTION where-clause violation
01402. 00000 - "view WITH CHECK OPTION where-clause violation"
```

Denying DML Operations

- You can ensure that no DML operations occur by adding the `WITH READ ONLY` option to your view definition.
- Any attempt to perform a DML operation on any row in the view results in an Oracle server error.

Denying DML Operations

```
CREATE OR REPLACE VIEW empvu10
    (employee_number, employee_name, job_title)
AS SELECT      employee_id, last_name, job_id
   FROM        employees
   WHERE       department_id = 10
   WITH READ ONLY ;
```

CREATE OR REPLACE VIEW succeeded.

Denying DML Operations (continued)

Any attempt to remove a row from a view with a read-only constraint results in an error:

```
DELETE FROM empvu10
WHERE employee_number = 200;
```

Similarly, any attempt to insert a row or modify a row using the view with a read-only constraint results in the same error.

Error report:
SQL Error: ORA-42399: cannot perform a DML operation on a read-only view

Removing a View

- You can remove a view without losing data because a view is based on underlying tables in the database

```
DROP VIEW view;
```

```
DROP VIEW empvu80;
```

```
DROP VIEW empvu80 succeeded.
```

What is a Sequence

A sequence:

- Automatically generates unique numbers
- Is a sharable object
- Is typically used to create a primary key value
- Replaces application code
- Speeds up the efficiency of accessing sequence values when cached in memory

Syntax for CREATE SEQUENCE

Define a sequence to generate sequential numbers automatically

```
CREATE SEQUENCE sequence
    [INCREMENT BY n]
    [START WITH n]
    [{MAXVALUE n | NOMAXVALUE}]
    [{MINVALUE n | NOMINVALUE}]
    [{CYCLE | NOCYCLE}]
    [{CACHE n | NOCACHE}];
```

Creating a Sequence

- Create a sequence named DEPT_DEPTDID_SEQ to be used for the primary key of the DEPARTMENTS table
- Do not use the CYCLE option

```
CREATE SEQUENCE dept_deptid_seq  
INCREMENT BY 10  
START WITH 120  
MAXVALUE 9999  
NOCACHE  
NOCYCLE;
```

Confirming Sequences

- Verify your sequence values in the USER_SEQUENCES data dictionary table

```
SELECT    sequence_name, min_value, max_value,  
          increment_by, last_number  
FROM      user_sequences;
```

- The LAST_NUMBER column display the next available sequence number if NOCACHE is specified

NEXTVAL and CURRVAL Pseudo columns

- NEXTVAL returns the next available sequence value. It returns a unique value every time it is referenced, even for different users
- CURRVAL obtains the current sequence value
- NEXTVAL must be issued for that sequence before CURRVAL contains a value

Using a Sequence

- Insert a new department named “Support” in location ID 2500

```
INSERT INTO departments (department_id,  
                        department_name, location_id)  
VALUES (dept_deptid_seq.NEXTVAL,  
        'Support', 2500);  
1 row created.
```

- View the current value for the DEPT_DEPTID_SEQ sequence

```
SELECT dept_deptid_seq.CURRVAL  
FROM dual;
```

Using a sequence

- Caching sequence values in memory gives faster access to those values
- Gaps in sequence values can occur when
 - A rollback occurs
 - The system crashes
 - A sequence is used in another table
- If the sequence was created with NOCACHE, the next available value, by querying the USER_SEQUENCES table

Modifying a Sequence

Change the increment value, maximum value, minimum value, cycle option, or cache option

```
ALTER SEQUENCE dept_deptid_seq  
        INCREMENT BY 20  
        MAXVALUE 999999  
        NOCACHE  
        NOCYCLE;
```

Sequence altered.

Guideline for Modifying Sequence

- You must be the owner or have the ALTER privilege for the sequence
- Only future sequence numbers are affected
- The sequence must be dropped and re-created to restart the sequence at a different number.
- Some validation is performed

Removing a Sequence

- Remove a sequence from the data dictionary by using the DROP SEQUENCE statement
- Once removed, the sequence can no longer be referenced.

```
DROP SEQUENCE dept_deptid_seq;  
Sequence dropped.
```

What is an index?

An index:

- Is a schema object
- Is used by the Oracle Server to speed up the retrieval of rows by using a pointer
- Can reduce disk I/O by using a rapid path access method to locate data quickly
- Is independent of the table it indexes
- Is used and maintained automatically by Oracle Server

How Are Indexes Created?

- Automatically: A unique index is created automatically when you define a PRIMARY KEY or UNIQUE constraint in a table definition
- Manually: Users can create nonunique indexes on columns to speed up access to the rows



Creating an Index

- Create an index on one or more columns

```
CREATE INDEX index  
ON table (column[, column]...);
```

- Improve the speed of query access to the LAST_NAME column in the EMPLOYEES table

```
CREATE INDEX emp_last_name_idx  
ON          employees(last_name);  
Index created.
```

When to create an Index

You should create an index if:

- A column contains a wide range of values
- A column contains a large number of null values
- One or more columns are frequently used together in a WHERE clause or a join condition
- The table is large, and most queries are expected to retrieve less than 2 to 4 percent of the rows.

When not to create an index

It is usually not worth creating an index if:

- The table is small
- The columns are not often used as a condition in the query
- Most queries are expected to retrieve more than 2 to 4 percent of the rows in the table
- The table is update frequently
- The indexed columns are referenced as part of an expression

Confirming Indexes

The USER_INDEXES data dictionary view contains the name of the index and its uniqueness

The USER_IND_COLUMNS view contains the index name, the table name, and the column name.

```
SELECT      ic.index_name, ic.column_name,
            ic.column_position col_pos, ix.uniqueness
FROM        user_indexes ix, user_ind_columns ic
WHERE       ic.index_name = ix.index_name
AND         ic.table_name = 'EMPLOYEES';
```

Function-Based Index

A function-based Index is an index based on expressions.

The index expression is built from table columns, constants, SQL functions, and user-defined functions.

```
CREATE INDEX upper_dept_name_idx  
ON departments (UPPER(department_name)) ;
```

Index created.

```
SELECT *  
FROM   departments  
WHERE  UPPER(department_name) = 'SALES' ;
```

Removing an Index

Remove an index from the data dictionary by using the DROP INDEX command

```
DROP INDEX index;
```

Remove the UPPER_LAST_NAME_IDX index from the data dictionary.

```
DROP INDEX upper_last_name_idx;  
Index dropped.
```

To drop an index, you must be the owner of the index or have the DROP ANY INDEX privilege

Summary

In this lesson, you should be have learned how to:

- Automatically generate sequence numbers by using a sequence generator
- View sequence information in the USER_SEQUENCES data dictionary
- Create indexes to improve query retrieval speed
- View index information in the USER_INDEXES dictionary table